Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3626641.3626949

RESEARCH-ARTICLE

Performance Analysis of 3D Rendering Method on Web-Based Augmented Reality Application Using WebGL and OpenGL Shading Language

MULTOHADI HAMZATURRAZAK, Brawijaya University, Malang, East Java, Indonesia

ERIQ MUHAMMAD JONEMARO, Brawijaya University, Malang, East Java, Indonesia

ARYO PINANDITO, Brawijaya University, Malang, East Java, Indonesia

Open Access Support provided by:

Brawijaya University

PDF Download
3626641.3626949.pdf
29 January 2026
Total Citations: 5
Total Downloads: 221

Published: 24 October 2023

Citation in BibTeX format

SIET 2023: International Conference on Sustainable Information Engineering and Technology
October 24 - 25, 2023
Badung, Bali, Indonesia

SIET '23: Proceedings of the 8th International Conference on Sustainable Information Engineering and Technology (October 2023)
https://doi.org/10.1145/3626641.3626949
ISBN: 9798400708503

Performance Analysis of 3D Rendering Method on Web-Based Augmented Reality Application Using WebGL and OpenGL Shading Language

Multohadi Hamzaturrazak
Eriq Muhammad Adams Jonemaro
hadigone@student.ub.ac.id
eriq.adams@ub.ac.id
Informatics Engineering Department
Faculty of Computer Science
Universitas Brawijaya
Malang, Indonesia

Aryo Pinandito*
aryo@ub.ac.id
Information System Department
Faculty of Computer Science
Universitas Brawijaya
Malang, Indonesia

ABSTRACT

In recent years, web programming has increased to the extent where it is capable of rendering 3D objects on the web. Its rendering performance varied depending on where it ran and what library was used to render the objects. This study aims to investigate the rendering performance of several 3D renderer libraries that render 3D objects in an augmented reality application. WebGL is one of the APIs that uses the Javascript programming language to render 3D objects in web applications. To create augmented reality on the web, a JavaScript library called AR.js is needed, which allows developers to create augmented reality on the web without the need to use additional plugins. AR.js supports various types of augmented reality, i.e., bookmarks, images, and location-based types. It also allows developers to incorporate visual and audio effects into augmented reality. This study aims to investigate the rendering performance of WebGL raw shaders and GLSL shaders in an HTML-based augmented reality application. The results suggested that both WebGL and GLSL could render low-poly and medium-poly 3D objects well. However, according to the test, WebGL performance drops significantly when rendering high-polygon 3D objects. Nonetheless, the GLSL library could render the same high-poly 3D objects at a similar level with native library rendering performance. Thus, suggest GLSL shaders as the recommended library to render 3D objects on the web.

CCS CONCEPTS

• **Computing methodologies → Rendering**; **Visibility**; **Shape analysis**; **Computer graphics**; **Graphics systems and interfaces**; **Mixed / augmented reality**.

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SIET 2023, October 24–25, 2023, Badung, Bali, Indonesia
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0850-3/23/10…$15.00
https://doi.org/10.1145/3626641.3626949

KEYWORDS

3D graphics, augmented reality, computer graphics, HTML5, texture mapping, WebGL

ACM Reference Format:
Multohadi Hamzaturrazak, Eriq Muhammad Adams Jonemaro, and Aryo Pinandito. 2023. Performance Analysis of 3D Rendering Method on Web-Based Augmented Reality Application Using WebGL and OpenGL Shading Language. In *International Conference on Sustainable Information Engineering and Technology (SIET 2023), October 24–25, 2023, Badung, Bali, Indonesia.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3626641.3626949

1 INTRODUCTION

Augmented Reality (AR) is a technology that allows users to see virtual objects placed in the real environment through devices such as cameras or smart glasses. AR technology has diverse possible uses across various sectors, such as education, gaming, commerce, and beyond [13, 19, 28]. AR development confronts issues with adapting to various platform environments. Harnessing the power of HTML, it becomes feasible to create AR content that is viewable via web browsers [17]. The versatile nature of HTML enables it to function as a springboard for innovative AR ventures. The primary advantage of using HTML for AR development is its portability [18]. HTML-based AR applications can function across a range of devices without necessitating supplementary programs [4].

Web browsers supporting HTML allow users to seamlessly interact with AR interfaces. With HTML, the ease of understanding its coding enables expedited learning, and AR content can now be easily crafted by developers. Moreover, HTML provides support for multiple elements like images, text, audio, video, and links, thereby enhancing the user experience in AR. Standard HTML elements provide basic Internet features. Being an industry-wide norm, HTML provides compatibility for AR apps across multiple platforms that support web technologies [30]. By allowing AR apps to be distributed across multiple devices and operating systems without restrictions, their reach and utilization are greatly increased [23].

HTML's capabilities are hindered when attempting to generate three-dimensional (3D) graphics that appear genuine and authentic [27]. Direct 3D rendering capabilities are not inherent in HTML code [1]. The fundamental premise of HTML rests upon a document framework that outlines the organizational hierarchy of components within a webpage, including textual content, visual elements,

connections, and interactive elements. The focus of this language is on representing content and organizing it effectively, incorporating dynamic aspects through links, forms, and JavaScript. In contrast to HTML not having innate capabilities for 3D rendering, it may nonetheless be paired with other tools to generate dynamic 3D visual elements on websites. One common approach is to utilize Web Graphics Library (WebGL), which serves as the web standard for rendering 3D graphics within browsers and acts as a solution to enhance the quality and speed of rendering virtual objects in HTML-based AR applications [9, 12].

By utilizing OpenGL Shading Language (GLSL) based on WebGL, the fast rendering method of AR HTML can overcome the limitations of HTML in rendering complex and realistic 3D graphics. WebGL is a 3D graphics Application Programming Interface (API) that enables the use of shaders to directly control the rendering process on a Graphics Processing Unit (GPU) [15]. With shaders, developers can create more advanced and diverse graphic effects compared with non-shader methods [11]. GLSL is a rendering method that implements custom vertex and fragment shaders for increased flexibility and control in the rendering process, as well as for creating complex visual effects [6, 7].

The study aims to investigate the performance of two shaders, WebGL and GLSL, in HTML-based AR applications. The experiments involve several scenarios of various levels of complexity and interaction. The rendering performance of WebGL and GLSL forms will be evaluated for the use of 3D in HTML-based AR. This study suggested the use of WebGL raw shader or GLSL in developing HTML-based AR applications.

2 LITERATURE REVIEW

2.1 HTML-based AR

A study [20] discussed the design and development of mobile AR applications using HTML-based AR technology that show the potential of web technology to enhance AR applications, shedding light on the strengths and limitations of this approach. To build AR applications using web technology, developers can utilize various technologies, e.g., WebGL, Three.js, and AR.js. These technologies enable developers to render 3D objects effectively and with adequate performance by leveraging the GPU available on the device [21, 24, 25].

Another study [2] suggested that the WebGL library took advantage of hardware acceleration available on a device. By using WebGL 3D technology on the web, developers have the possibility of accessing GPUs for 3D rendering. To use WebGL requires a library that can access it; the library is called Three.js. Theree.js is a library made of JavaScript to create interactive 3D applications on the web. Further, to add AR to the web, a library is needed to facilitate the creation of AR applications, i.e., AR.js. By using AR.js, AR creation on the web can be made without the need for additional plugins or applications [26].

Using AR technology on the web has the advantage of allowing developers to construct AR applications easily and quickly, and web technology allows the developer to build AR applications that can be accessible on a range of devices. However, there are flaws in applying this AR technology, namely when producing 3D objects or having it supported by WebGL for rendering, yet its rendering capabilities are still limited in contrast to native rendering technology. Furthermore, developers must consider compatibility across different browsers and consumer devices. By applying such technology, it is expected that developers understand the advantages and challenges of using web technology. Besides, this research can inspire and provide ideas for the expansion of web-based AR applications.

2.2 GPU Rendering on HTML Web Page

The quality of rendering a 3D object in the context of HTML-based AR becomes particularly challenging because it needs rendering within a web browser, which has constraints on optimizing GPU performance. A prior study [8] discussed the rendering performances of AR-based applications and suggested the use of WebGL-based shader technology to improve the performance of rendering 3D on the web. This study may suggest supporting insights for the development of 3D HTML applications and additional research on web applications that employ shader libraries.

GLSL is a programming language used in web application development that handles 3D rendering using native hardware GPUs. Additionally, GLSL serves as a shader renderer that controls how objects or images are rendered on a device's screen. Because GLSL has the capacity to efficiently utilize the resources of the devices it runs on, it may efficiently render and increase the performance of rendering 3D objects in web-based AR applications.

Another study [5] analyzed the strategy of WebGL-based 3D visualization for large-scale data, providing insights into performance analysis that are relevant to the context of this study. In the study, it was shown that the methods derived from GLSL have provided fairly good control and flexibility in rendering 3D objects and creating complex visual effects. Another study [10] also investigated the rendering performance of WebGL and WebGL-based GPUs; hence, it supports this study.

In the development of HTML-based AR applications, the use of GLSL can improve 3D rendering performance by optimizing the utilization of GPU resources on computer devices. Such methods allow users to provide more flexible control over the rendering process. Thus, the utilization of GLSL in a web application is suggested to offer a better user experience.

2.3 WebGL Raw Shader and GLSL Shader

It is known that GLSL shaders could deliver better rendering performance than WebGL raw shaders. GLSL's helped applications optimize the shader's code, thus generating faster and more efficient performance. Furthermore, using a GLSL shader enables the development of complex rendering features, such as realistic lighting and captivating visual effects.

Several studies have been conducted on the performance of WebGL and GLSL [16, 31]. Several experiments have found that using shaders can greatly enhance rendering performance when compared to using solely WebGL shaders. However, they have not properly explored the effects of shader and GLSL shader usage characteristics. Furthermore, prior experiments have not carried out analysis in rendering high-polygon 3D objects for AR applications; hence, they were investigated in this study.

This study focuses on investigating the influence of the characteristics of using WebGL raw shaders and GLSL shaders on rendering high-poly AR 3D web objects. The main objective of this study is to prove the hypothesis that employing GLSL shaders in web-based AR applications could improve the application's rendering performance.

2.4 Difference between Native Shader and WebGL Shader

The web 3D shaders differ from the native application in terms of rendering efficiency. AR applications that use web technology and try to access hardware, particularly GPUs, are obstructed by an extra layer of abstraction barrier that resides in the web browser [14]. Because native programs could directly access the device's GPU, it is natural that 3D shaders employed in a native application have better rendering performance.

In addition to the ability of native applications to access GPUs directly, they are also able to access other hardware such as sensors and cameras, thus optimizing the resource use of devices and achieving optimal performance [29]. Unlike native applications, web applications may still have access to hardware features, but with limited capabilities.

However, in terms of compatibility and portability, HTML-based AR applications can be accessed through web browsers on a variety of devices, such as smartphones, tablets, and personal computers. It provides easy access to a wider range of devices without having to have it installed on the device. On the other hand, native AR applications require a dedicated platform to access the application and have to be installed on the platform directly.

3 METHODOLOGY

This study follows the research methodology as shown in Fig. 1. The study begins with research problems and identification activities, which lead to designing an experiment to solve the specified research questions. To answer the research questions, two HTML5-based mobile AR applications were designed and developed. The applications were used to measure the 3D rendering performance of using two different rendering methods in HTML5-based AR applications, i.e., WebGL and GLSL. The measured rendering performance data were then collected in the experiment and analyzed in this study.

3.1 System Architecture and Research Problem Identification

The AR system architecture of this study is implemented as shown in Fig. 2. Because this applies a client-server architecture to deliver the AR application, client computers do not need to perform the installation of the AR application but rather make requests and receive responses directly from the web server. This research uses several programming languages in the context of software development, with special emphasis on HTML, JavaScript, and the GL Shading Language. Additionally, the following hardware specifications were used in the study: a laptop computer with a 7th generation Intel Core i7 processor and an Nvidia 950M GPU. The application requests the required 3D models according to a particular AR marker and renders the 3D models on the device's screen. In the design, both WebGL and GLSL rendering methods are used in the AR application to render the specified 3D objects on the computers.

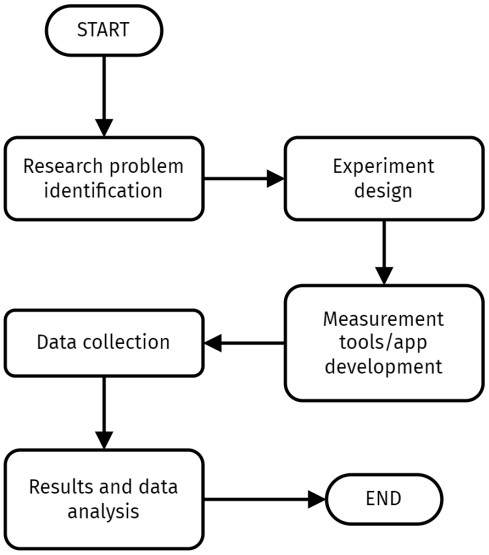


Figure 1: Research Methodology

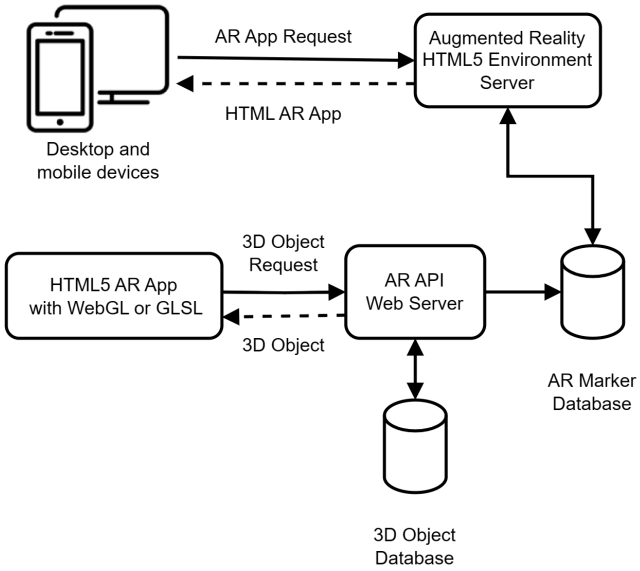


Figure 2: System Architecture

The AR application is supposedly displaying 3D objects when a particular marker is detected by the device's camera. The rendering engine of the application will render the expected 3D models on the screen in place of the detected marker. The 3D rendering engine that can be used to render 3D objects is WebGL. WebGL consists of two main shader components, i.e., the WebGL raw shader and the GLSL shader, whose performance is evaluated in this study.

Additionally, their rendering performance was measured at three different quality levels of 3D objects, i.e., low-poly, medium-poly, and high-poly 3D objects.

3.2 Measurement

In Fig. 3, the procedure for conducting an experiment using three shaders is described. Two of the shaders are WebGL shaders, namely the raw shader from WebGL and GLSL, while the third one is a native shader as the performance evaluation baseline. The purpose of this experiment is to observe the performance of rendering 3D objects using these three shaders and measure their performance. After the completion of the 3D object rendering process, the rendering process is measured and analyzed to evaluate the performance of the shaders in both web and native environments. In the experiment, several shaders will be tested, and each shader will be applied to three different 3D objects. Each 3D object has three different polygon counts, as shown in Fig. 4.

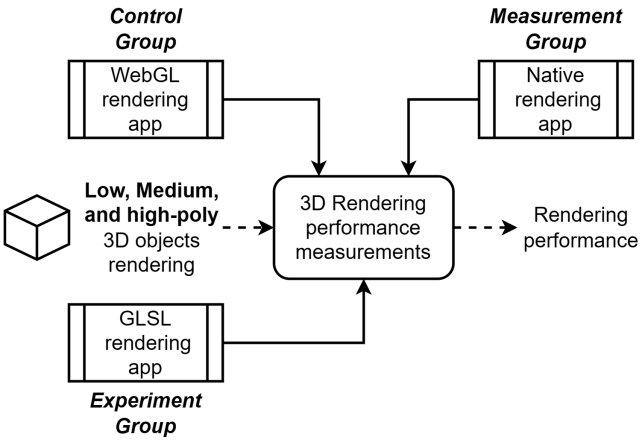
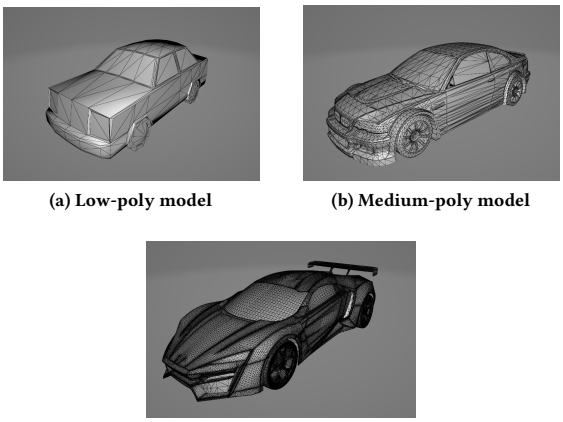


Figure 3: Experiment Flow

Each object rendered by the AR application, as shown in Fig. 4, is a 3D model of a car. The models consisted of 2,000 polygons, 51,000 polygons, and 4,000,000 polygons for the low-polygon, medium-polygon, and high-polygon 3D objects, respectively [3, 22]. During the performance measurement process in the experiment, each 3D object will be rotated vertically, horizontally, and diagonally. The performance of polygon rendering will be recorded when the object returns to its initial rotation position. The measurements are performed 30 times for each object using three respective 3D rendering engines, i.e., WebGL, GLSL, and native shaders.

3.3 Data Collection

The measurement gathers data related to the application's rendering speed in frames per second (fps) and its completion rendering time in milliseconds (ms) for each AR application using WebGL, GLSL, and native shaders. The data is analyzed further to determine the performance of AR applications when using WebGL, GLSL shaders, and native renderers to render 3D objects. The data will be organized into their respective categories and analyzed quantitatively to provide detailed information regarding the rendering performance and efficiency of AR applications using WebGL and GLSL.



(a) Low-poly model **(b) Medium-poly model**

(c) High-poly model

Figure 4: 3D object models used in the experiment.

3.4 Data Analysis

Before analyzing the data, the Shapiro-Wilk test is conducted on the data to investigate whether the obtained data conform the assumption of a normal distribution. Non-parametric analysis, e.g., the Mann-Whitney U test, will be used to compare data between each group when they do not qualify for parametric analysis.

The data are grouped into several categories, each consisting of the polygon level and object movement for each shader used to investigate whether the polygon level of the 3D objects and their movement affected the shader's rendering performance in HTML-based AR applications. Lastly, a comparison analysis using either a parametric T-test or a non-parametric Mann-Whitney U test will be conducted to investigate the performance of HTML-based and native AR applications that use WebGL and GLSL shaders.

4 RESULT AND ANALYSIS

4.1 Measurement Data

According to the experiment design, there are three types of shaders used to render the 3D models. i.e., raw shaders of WebGL, GLSL, and native shaders. The performance data for each shader is taken 90 times—that is, 30 times for each spinning direction, i.e., horizontally, vertically, and diagonally. Each run measures its rendering speed (S_R) and rendering time (T_R), for a total of 1,620 measurements. Several samples of the measurement results are shown in Table 1.

4.2 Analysis

The obtained measurement data are then categorized based on the type of experiment, shader type, and polygon level. The descriptive statistics summary of the rendering speed and time are shown in Table 2 and Table 3, respectively.

Before conducting comparative statistical tests and analyses, the native data is set aside as a reference. Subsequently, the shader data from WebGL, whether it is raw WebGL shader and GLSL, that has

Table 1: Shader Rendering Performance

Shader	Polygon Level	Movement	S_R (fps)	T_R (ms)
WebGL	Low	Horizontal	60	16
WebGL	Medium	Vertical	60	16
WebGL	High	Diagonal	56	20
GLSL	Low	Horizontal	61	16
GLSL	Medium	Vertical	60	16
GLSL	High	Diagonal	58	17
Native	Low	Horizontal	60	16
Native	Medium	Vertical	60	16
Native	High	Diagonal	60	16

S_R: Rendering Speed, T_R: Rendering Time

Table 2: Descriptive Statistics of Performance Measurement rendering speed

Shader	Polygon	n	mean	sd
WebGL	Low	30	60.09	0.286
WebGL	Medium	30	60	0
WebGL	High	30	50.6	4.44
GLSL	Low	30	60.56	0.500
GLSL	Medium	30	60.58	0.497
GLSL	High	30	57.96	2.49
Native	Low	30	60	0
Native	Medium	30	60	0
Native	High	30	60	0

Table 3: Descriptive Statistics of Performance Measurement rendering time

Shader	Polygon	n	mean	sd
WebGL	Low	30	16	0
WebGL	Medium	30	16	0
WebGL	High	30	19.3	2
GLSL	Low	30	16	0
GLSL	Medium	30	16	0
GLSL	High	30	17	0.6
Native	Low	30	16	0
Native	Medium	30	16	0
Native	High	30	16	0

been obtained is tested for its normal distribution. The results of the normality test using the Shapiro-Wilk method for performance data are shown in Table 4. Since all the p-values for WebGL shader from the Shapiro-Wilk test are less than the determined significance level of 0.05, further analyses will be conducted using non-parametric approaches.

The results of the Mann-Whitney U test on the performance data for rendering speed can be shown in Table 5. According to the analysis regarding rendering speed, it can be said that there

Table 4: The Shapiro-Wilk Test for Normality

Shader	Polygon	S_R *p-value*	T_R *p-value*
WebGL	Low	$<2.2\times10^{-16}$	-
WebGL	Medium	-	-
WebGL	High	0.001029	1.324×10^{-05}
GLSL	Low	1.094×10^{-13}	-
GLSL	Medium	9.009×10^{-14}	-
GLSL	High	4.105×10^{-8}	1.368×10^{-08}

is a significant difference in rendering speed between the scenarios that use WebGL and GLSL shaders where the p-value of the Mann-Whitney U test is less than the significance level of 0.05. Furthermore, when rendering low, medium, and high polygon 3D models, the rendering performance of each application group also shows a statistically different result, as shown in Table 5. The results of the Mann-Whitney U test on the performance data regarding rendering time also suggested that GLSL shaders render high-poly 3D models faster than WebGL shaders, as shown in Table 5.

Table 5: Mann-Whitney U-Test for Non-Parametric Performance Measurement Between WebGL and GLSL

Polygon	*p-value*	WebGL	GLSL	Metric
Low	2.417×10^{-11}	60.09	60.56	S_R
Medium	2.2×10^{-16}	60	60.58	S_R
High	2.2×10^{-16}	50.6	57.96	S_R
Low	-	16	16	T_R
Medium	-	16	16	T_R
High	2.2×10^{-16}	19	17	T_R

Based on the results, it is possible to conclude that there is a difference in terms of rendering speed between WebGL and GLSL shaders in rendering high-poly 3D models. However, all shaders were able to render low-poly and medium-poly 3D models similarly.

According to the comparison results as shown in Table 6, it can be suggested that WebGL shaders show their good performance at 60 fps in rendering low and medium polygon 3D models but show their struggle in rendering high-poly models. Similar trends regarding rendering time can also be seen in Table 6, where WebGL shows their struggles to render high-poly 3D models at a slower speed to render a full movement of the models.

The comparison summary regarding the rendering speed and rendering time performance of WebGL, GLSL, and native shaders at three different polygon levels, i.e., low, medium, and high poly, is shown in Fig. 5 and Fig. 6, respectively.

According to the findings, in terms of rendering speed performance, native shaders, WebGL raw shaders, and GLSL show great performance in low and medium polygon 3D models at the recommended rendering speed of 60 fps. However, in rendering high-polygon 3D models, WebGL and GLSL shaders show reduced performance, even though in some situations, GLSL shaders could attain

Table 6: Measurement of rendering time Between WebGL, GLSL and Native

Polygon	WebGL	GLSL	Native	Metric
Low	60.1	60.56	60	S_R
Medium	60	60.58	60	S_R
High	50.6	57.97	60	S_R
Low	16	16	16	T_R
Medium	16	16	16	T_R
High	19.3	17	16	T_R

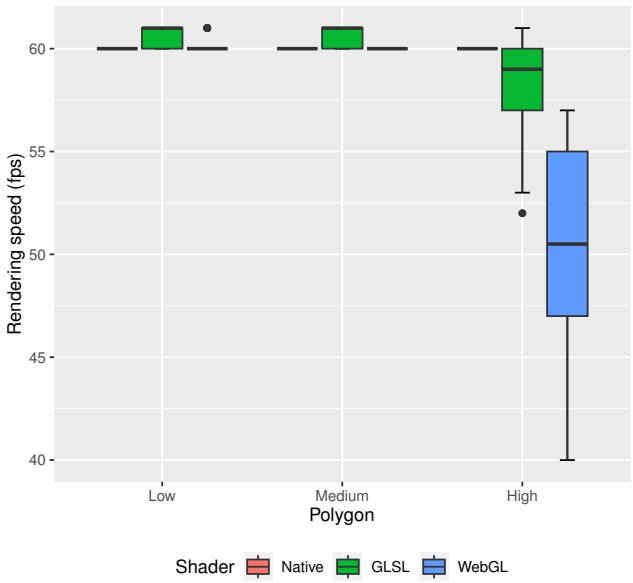


Figure 5: Rendering speed performance

60 fps rendering speed. At all times, the native renderer remains consistent at 60 frames per second across all polygon levels of 3D models. This implies that GLSL could achieve similar performance with a native 3D renderer.

In terms of rendering time, WebGL and GLSL could render in a similar way with a native renderer on low- and medium-polygon 3D models. However, when rendering high-polygon 3D models, GLSL starts to show slower rendering times but is still better than WebGL. Overall, the WebGL raw shader and GLSL showed good performance in terms of rendering speed and rendering time at both low and medium poly levels but experienced a decrease in performance in rendering high-polygon 3D models. Nevertheless, GLSL is able to deliver similar performance as the native renderer.

5 LIMITATION

This study used an empirical approach to measure and evaluate the performance of the WebGL raw shader and the GLSL shader in rendering 3D objects for an AR web-based application. Three different objects with three different numbers of polygons were used in the measurement. According to the measurement results of

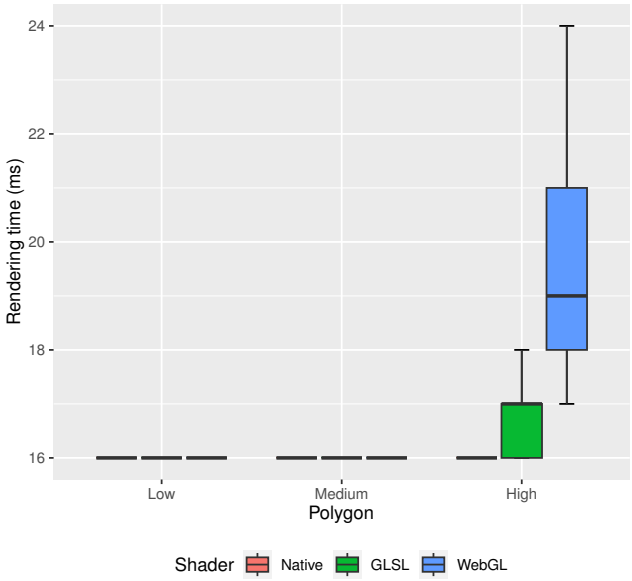


Figure 6: Rendering time performance

the low and medium poly models, it seems that both the WebGL raw shader and the GLSL shader limit the 3D rendering framerate to around 60 fps. Thus, even though the difference between the performance of WebGL and GLSL shaders was statistically different, the actual measured performance difference may be different in its actual use.

In this study, the AR application renders only the 3D object on the screen. Depending on the device's 3D hardware capability, simultaneously rendering more than one 3D object or adding virtual 3D environments or lighting effects may affect the overall rendering performance significantly.

6 CONCLUSION

According to the analysis results, WebGL raw shader and GLSL perform differently in rendering high-polygon 3D models. While both WebGL and GLSL excel at rendering low and medium polygon models at a consistent 60 fps recommended performance level, they start to show a decreasing performance in rendering high polygon models. Additionally, WebGL raw shader and GLSL perform similarly with the native renderer in rendering low and medium poly models and start to struggle when rendering high poly models.

Overall, both WebGL and GLSL show good performances in terms of rendering speed and rendering time at low and medium polygon models, even though they start to show a decrease in rendering performance at medium and high polygon models. According to the investigation results, the GLSL shader has similar performance to the native renderer. Thus, it can be suggested as the recommended renderer to use in web applications that render 3D models, such as in an AR application.

ACKNOWLEDGMENTS

This study is supported by the Master's of Computer Science Program of The Faculty of Computer Science, Universitas Brawijaya.

REFERENCES

[1] J.R. Alder and S.W. Hostetler. 2015. Web based visualization of large climate data sets. *Environmental Modelling and Software* 68 (2015), 175–180. https://doi.org/10.1016/j.envsoft.2015.02.016

[2] K. Andrews and Benedict Wright. 2014. FluidDiagrams: Web-Based Information Visualisation using JavaScript and WebGL. (2014). https://doi.org/10.2312/eurovisshort.20141155

[3] Pooja Bhawar, Nitin Ayer, and Sameer Sahasrabudhe. 2013. Methodology to Create Optimized 3D Models Using Blender for Android Devices. In *2013 IEEE Fifth International Conference on Technology for Education (t4e 2013)*. 139–142. https://doi.org/10.1109/T4E.2013.41

[4] D. Bridie and J. Santell. [n. d.]. Build an augmented reality (AR) app using the WebXR Device API. Available at https://codelabs.developers.google.com/ar-with-webxr (2021/06/25).

[5] Sören Discher, Rico Richter, and Jürgen Döllner. 2018. A Scalable WebGL-Based Approach for Visualizing Massive 3D Point Clouds Using Semantics-Dependent Rendering Techniques. In *Proceedings of the 23rd International ACM Conference on 3D Web Technology* (Poznań, Poland) *(Web3D '18)*. Association for Computing Machinery, New York, NY, USA, Article 19, 9 pages. https://doi.org/10.1145/3208806.3208816

[6] MDN Web Docs. 2021. GLSL Shaders. Retrieved June 6, 2021 from https://developer.mozilla.org/en-US/docs/Games/Techniques/3D_on_the_web/GLSL_Shaders

[7] MDN Web Docs. 2021. WebGLRenderingContext. Retrieved June 6, 2021 from https://developer.mozilla.org/en-US/docs/Web/API/WebGLRenderingContext/compileShader

[8] J Ferrão, P Dias, BS Santos, and M Oliveira. 2023. Environment-Aware Rendering and Interaction in Web-Based Augmented Reality. *Journal of Imaging* (2023). https://doi.org/10.2139/ssrn.4251267 Query date: 2023-06-22 16:28:38.

[9] E. Gobbetti, F. Marton, Marcos Balsa, F. Ganovelli, and Marco Di Benedetto. 2012. Adaptive quad patches: an adaptive regular structure for web distribution and adaptive rendering of 3D models. (2012), 9–16. https://doi.org/10.1145/2338714.2338716

[10] Brahim Youssouf Goukouni, Muhammad Aamir, Waqar Ali, Zaheer Ahmed Dayo, Waheed Ahmed Abro, Muhammad Ishfaq, and Guan Yurong. 2022. Methods Tested to Optimize the Performance of WEBGL Applications. In *Sensing Technology*, Nagender Kumar Suryadevara, Boby George, Krishanthi P. Jayasundera, Joyanta Kumar Roy, and Subhas Chandra Mukhopadhyay (Eds.). Springer International Publishing, Cham, 339–354.

[11] K Halladay. 2019. *Practical Shader Development: Vertex and Fragment Shaders for Game Developers*. books.google.com. https://books.google.com/books?hl=en&lr=&id=ouORDwAAQBAJ&oi=fnd&pg=PR5&dq=practical+shader+development+vertex+and+fragment+shaders+for+game+developers&ots=ZLPk8J_eCL&sig=mbAffq7bDe3069couq7p2ShkGBk

[12] J. Jankowski, Sandy Ressler, K. Sons, Yvonne Jung, J. Behr, and P. Slusallek. 2013. Declarative integration of interactive 3D graphics into the world-wide web: principles, current approaches, and research agenda. (2013), 39–45. https://doi.org/10.1145/2466533.2466547

[13] Ali Zabihi Kolaei, Erfan Hedayati, Mostafa Khanzadi, and Gholamreza Ghodrati Amiri. 2022. Challenges and opportunities of augmented reality during the construction phase. *Automation in Construction* 143 (2022), 104586. https://doi.org/10.1016/j.autcon.2022.104586

[14] Man Hee Lee. 2006. Accelerating Depth Image-Based Rendering Using GPU. (2006). https://doi.org/10.1007/11848035_74

[15] Liang Li, Xiuquan Qiao, Qiong Lu, Pei Ren, and Ruibin Lin. 2020. Rendering Optimization for Mobile Web 3D Based on Animation Data Separation and On-Demand Loading. *IEEE Access* 8 (2020), 88474–88486. https://doi.org/10.1109/ACCESS.2020.2993613

[16] Ian McEwan, David Sheets, Mark Richardson, and Stefan Gustavson. 2012. Efficient Computational Noise in GLSL. *Journal of Graphics Tools* 16, 2 (2012), 85–94. https://doi.org/10.1080/2151237X.2012.649621 arXiv:https://doi.org/10.1080/2151237X.2012.649621

[17] J. Medley. [n. d.]. Augmented reality for the web. Available at https://developer.chrome.com/blog/ar-for-the-web/ (2018/06/15).

[18] Aryo Pinandito, Hanifah Muslimah Az-zahra, Lutfi Fanani, and Anggi Valeria Putri. 2017. Analysis of web content delivery effectiveness and efficiency in responsive web design using material design guidelines and User Centered Design. In *2017 International Conference on Sustainable Information Engineering and Technology (SIET)*. 435–441. https://doi.org/10.1109/SIET.2017.8304178

[19] Bibhu Prasad Mohanty and Laxmi Goswami. 2023. Advancements in augmented reality. *Materials Today: Proceedings* 81 (2023), 515–519. https://doi.org/10.1016/j.matpr.2021.03.696 International Virtual Conference on Sustainable Materials (IVCSM-2k20).

[20] Xiuquan Qiao, Pei Ren, Schahram Dustdar, Ling Liu, Huadong Ma, and Junliang Chen. 2019. Web AR: A Promising Future for Mobile Augmented Reality—State of the Art, Challenges, and Insights. *Proc. IEEE* 107, 4 (April 2019), 651–666. https://doi.org/10.1109/JPROC.2019.2895105

[21] Solange Gomes Santos and Jorge C. S. Cardoso. 2019. Web-based Virtual Reality with A-Frame. In *2019 14th Iberian Conference on Information Systems and Technologies (CISTI)*. 1–2. https://doi.org/10.23919/CISTI.2019.8760795

[22] V. Stojanovic, D. Blackwood, D. Gilmour, J.P. Isaacs, and R.E. Falconer. 2013. Comparison of Advanced and Standard Real-Time 3D Rendering Methods for Interactive Landscapes (Short Paper Version). In *2013 17th International Conference on Information Visualisation*. 539–544. https://doi.org/10.1109/IV.2013.73

[23] Toqeer Ali Syed, Muhammad Shoaib Siddiqui, Hurria Binte Abdullah, Salman Jan, Abdallah Namoun, Ali Alzahrani, Adnan Nadeem, and Ahmad B. Alkhodre. 2023. In-Depth Review of Augmented Reality: Tracking Technologies, Development Tools, AR Displays, Collaborative AR, and Security Concerns. *Sensors* 23, 1 (2023). https://doi.org/10.3390/s23010146

[24] AR.js Team. 2022. AR.js - Efficient Augmented Reality for the Web. https://ar-js-org.github.io/AR.js-Docs/.

[25] Three.js Team. 2022. Three.js - JavaScript 3D library. https://threejs.org/.

[26] WebGL Team. 2022. WebGL - OpenGL ES 2.0 for the Web. https://www.khronos.org/webgl/.

[27] Renato M. Toasa, Paúl Francisco Baldeón Egas, Miguel Alfredo Gaibor Saltos, Mateo A. Perreño, and Washington X. Quevedo. 2019. Performance Evaluation of WebGL and WebVR Apps in VR Environments. In *Advances in Visual Computing*, George Bebis, Richard Boyle, Bahram Parvin, Darko Koracin, Daniela Ushizima, Sek Chai, Shinjiro Sueda, Xin Lin, Aidong Lu, Daniel Thalmann, Chaoli Wang, and Panpan Xu (Eds.). Springer International Publishing, Cham, 564–575.

[28] Herman Tolle, Aryo Pinandito, EMA Jonemaro, and Kohei Arai. 2015. Virtual reality game controlled with user's head and body movement detection using smartphone sensors. *ARPN Journal of Engineering and Applied Sciences* 11, 20 (2015), 9776–9782.

[29] James White. 2013. Going native (or not): Five questions to ask mobile application developers. *The Australasian medical journal* (2013). https://doi.org/10.4066/AMJ.2013.1576

[30] Yue Yin, Pai Zheng, Chengxi Li, and Lihui Wang. 2023. A state-of-the-art survey on Augmented Reality-assisted Digital Twin for futuristic human-centric industry transformation. *Robotics and Computer-Integrated Manufacturing* 81 (2023), 102515. https://doi.org/10.1016/j.rcim.2022.102515

[31] Qi Zhang. 2019. Medical data visual synchronization and information interaction using Internet-based graphics rendering and message-oriented streaming. *Informatics in Medicine Unlocked* 17 (2019), 100253. https://doi.org/10.1016/j.imu.2019.100253